

BlazeDemo Automation – Capstone Project

GitHub Repository-

<https://github.com/Himanshu0320/Capstone-Project>

Project Type

Web Automation Testing

PROJECT OVERVIEW:

The BlazeDemo Flight Booking Automation Framework is a robust web automation project developed to automate the end-to-end flight booking workflow of the BlazeDemo application. The framework validates major functionalities including flight search, flight selection, passenger information submission, and booking confirmation.

The framework follows the Page Object Model (POM) design pattern, ensuring maintainability, reusability, scalability, and easy execution across multiple browsers.

The framework also supports Data-Driven Testing using Excel files and generates detailed execution reports using TestNG and Extent Reports.

Objectives of the Project:

The main objectives of this automation project are:

- Automate the complete flight booking workflow.
- Reduce repetitive manual testing activities.
- Improve test execution efficiency.
- Validate application functionality across multiple test data sets.
- Generate detailed execution reports.
- Demonstrate industry standard automation framework practices.

Technology Stack & Tools:

Category	Tools & Technologies Used
Language & Frameworks	Java, Selenium WebDriver, TestNG
Build & Architecture	Maven, Page Object Model (POM)
Data & Reporting	Apache POI (Excel), Extent Reports
CI/CD & DevOps	Git, GitHub, Jenkins, Docker
IDE	Eclipse IDE

Framework Architecture:

The automation framework follows the Page Object Model (POM) design pattern, which improves code reusability, maintainability, and scalability.

Project Structure

Base Package: - Contains browser initialization and teardown methods.

Pages Package: - Contains page classes representing application web pages.

It has multiple classes such as:

- HomePage
- FlightSelectionPage
- PurchasePage
- ConfirmationPage

Tests Package: - Contains all test execution classes.

- LaunchTest
- CrossBrowserTest

- SearchFlightTest
- FlightBookingTest

Utilities Package: - Contains reusable utility classes.

- ConfigReader
- ExcelUtil
- ScreenshotUtil
- ExtentManager

Listeners Package: - Contains TestNG listener implementation.

- TestListener

DataProviders Package

Signifies a Data Driven approach.

- BookingDataProvider

Resources

Contains test data and configuration files.

- PassengerData.xlsx
- config.properties

Test Scenarios Automated

Scenario 1: Application Launch

Description

Verify that the BlazeDemo application launches successfully, and the home page is displayed.

Steps

1. Launch the browser.
2. Navigate to the BlazeDemo URL.
3. Verify the page title.

Expected Result

The BlazeDemo home page should open successfully with the correct title.

Scenario 2: Search Flights

Description

Verify that users can search for available flights by selecting departure and destination cities.

Steps

1. Open the BlazeDemo application.
2. Select the departure city.
3. Select the destination city.
4. Click on Find Flights.

Expected Result

The list of available flights should be displayed successfully.

Scenario 3: Complete Flight Booking

Description

Verify that users can complete flight booking successfully.

Steps

1. Read passenger details from Excel.
2. Enter passenger information.
3. Enter payment details.
4. Click Purchase Flight.

Expected Result

The booking should be completed successfully.

Scenario 4: Validate Booking Confirmation

Description

Verify that the booking confirmation page is displayed after successful booking.

Steps

1. Complete the booking process.
2. Navigate to the confirmation page.
3. Validate the confirmation message.

Scenario 5: Data-Driven Testing

Description

Verify the flight booking functionality using multiple test data records stored in an Excel file.

Steps

1. Read multiple rows from the Excel sheet.
2. Execute the booking flow for each record.
3. Validate successful booking.

Expected Result

All Excel data sets should execute successfully without errors.

Scenario 6: Cross Browser Testing

Description

Verify that the application works correctly on multiple browsers.

Expected Result

The application should function correctly and tests should pass on all supported browsers.

6. Reporting Mechanism

The framework generates multiple reports for execution tracking and analysis.

TestNG Report

Location:

test-output/index.html

Features:

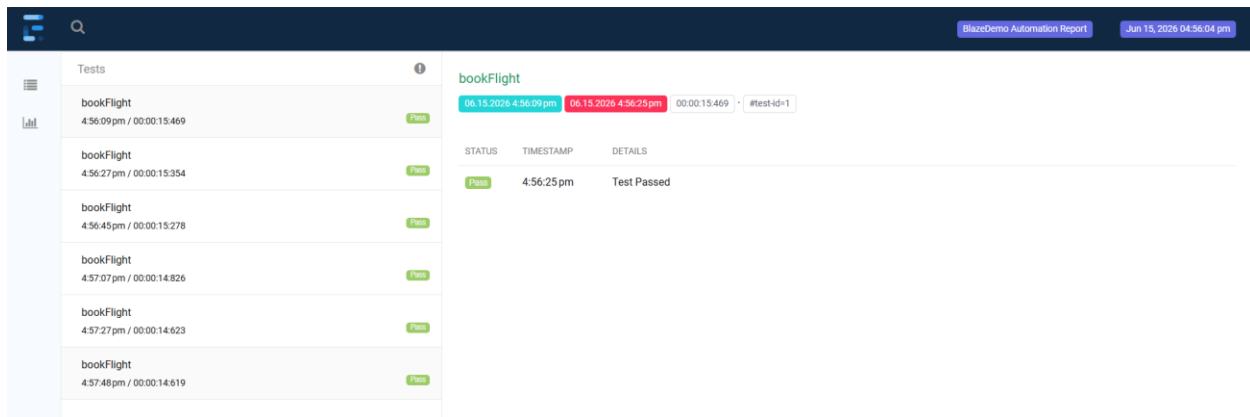
- Total Tests Executed

- Passed Tests
- Failed Tests
- Skipped Tests
- Execution Summary

Extent Report

Location:

Reports/ExtentReport.html



The screenshot displays the Extent Report interface. On the left, a sidebar shows a list of tests under the heading 'Tests'. The main area shows a detailed view of a test named 'bookFlight'. The test status is 'Pass', and the execution time is 00:00:15.469. The test was executed on 06.15.2026 at 4:56:09 pm. The test details table shows a single row with the status 'Pass', the timestamp '4:56:25 pm', and the details 'Test Passed'.

Tests	bookFlight
bookFlight 4:56:09 pm / 00:00:15.469	Pass
bookFlight 4:56:27 pm / 00:00:15.354	Pass
bookFlight 4:56:45 pm / 00:00:15.278	Pass
bookFlight 4:57:07 pm / 00:00:14.826	Pass
bookFlight 4:57:27 pm / 00:00:14.623	Pass
bookFlight 4:57:48 pm / 00:00:14.619	Pass

STATUS	TIMESTAMP	DETAILS
Pass	4:56:25 pm	Test Passed

Features:

- Interactive Dashboard
- Pass/Fail Status
- Execution Timeline
- Detailed Logs
- Test Execution Summary
- Screenshot Integration

Screenshot Capture Functionality

The framework automatically captures screenshots during execution on failures and success.

Screenshots are captured for:

- Flight Search Page
- Flight Selection Page
- Purchase Page
- Confirmation Page
- Failed Test Cases

Benefits:

- Easier defect analysis
- Improved reporting
- Better execution evidence

Jenkins Integration

The automation framework is integrated with Jenkins Pipeline to enable continuous integration.

Jenkins Pipeline Stages

1. Checkout Source Code
2. Clean Project
3. Build Project
4. Execute TestNG Test Cases
5. Data-Driven Test Execution
6. Generate Extent Reports
7. Publish Reports
8. Archive Build Artifacts

no artifacts found that match the file pattern reports/ / .

```
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] stage  
[Pipeline] { (Declarative: Post Actions)  
[Pipeline] echo
```

Pipeline Execution Completed

```
[Pipeline] echo
```

BlazeDemo Automation Build Successful

```
[Pipeline] }  
[Pipeline] // stage  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // withEnv  
[Pipeline] }  
[Pipeline] // node  
[Pipeline] End of Pipeline
```

Finished: SUCCESS

The screenshot displays the Jenkins web interface for a job named 'Capstone-Job', specifically for build #12. The top navigation bar shows the Jenkins logo, the job name, and build number. A green checkmark indicates the build is successful. Below the navigation bar, a summary bar shows the build was started by Himanshu Sharma, started 4 hours and 16 minutes ago, and took 2 minutes and 28 seconds to complete. A progress bar visualizes the pipeline stages: Start, Checkout SCM, Tool Install, Checkout Code, Clean Project, Compile Project, Execute TestNG Tests, Archive Reports, Post Actions, and End. All stages are marked with green checkmarks. Below the progress bar, a list of stages is shown on the left, and a detailed view of the 'Post Actions' stage is shown on the right. The 'Post Actions' stage includes two steps: 'Pipeline Execution Completed' (43ms) and 'BlazeDemo Automation Build Successful' (25ms). The interface also includes a search bar, a 'Rerun' button, and a 'Changes' link.

Jenkins / Capstone-Job / #12 / Stages

✓ < #12

Started by Himanshu Sharma Started 4 hr 16 min ago Queued 88 ms Took 2 min 28 sec <> Changes Artifacts

Start End

Checkout SCM Tool Install Checkout Code Clean Project Compile Project Execute TestNG Tests Archive Reports Post Actions

Search

✓ Checkout SCM 2s

✓ Tool Install 0.31s

✓ Checkout Code 2s

✓ Clean Project 3s

✓ Compile Project 5s

✓ Execute TestNG Tests 2m 9s

✓ Post Actions 0.11s Started 4h 14m ago Jenkins

✓ Pipeline Execution Completed 43ms

0 Pipeline Execution Completed

✓ BlazeDemo Automation Build Successful 25ms

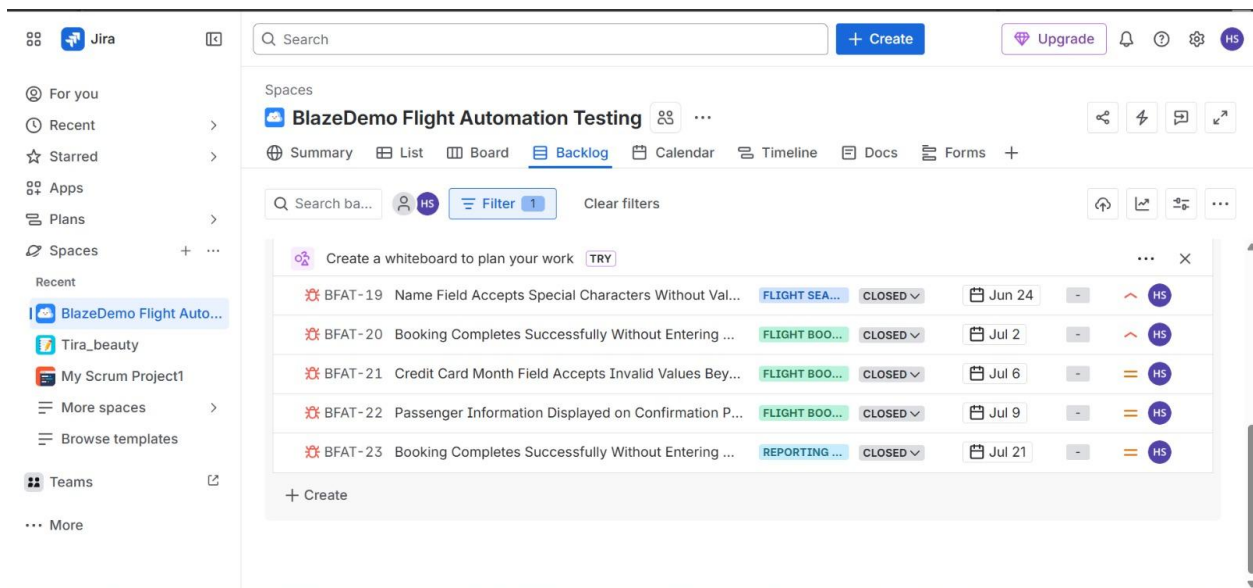
0 BlazeDemo Automation Build Successful

Benefits

- Automated execution
- Scheduled builds
- Faster feedback on code changes
- CI/CD implementation
- Easy monitoring of test results

Defects Identified During Testing

Several defects were identified during execution:



The screenshot shows a Jira interface with a sidebar on the left containing navigation options like 'For you', 'Recent', 'Starred', 'Apps', 'Plans', 'Spaces', and 'Teams'. The main content area displays a backlog for the 'BlazeDemo Flight Automation Testing' space. The backlog is filtered to show 1 item. The defects listed are:

Defect ID	Defect Description	Category	Status	Due Date	Assignee
BFAT-19	Name Field Accepts Special Characters Without Val...	FLIGHT SEA...	CLOSED	Jun 24	HS
BFAT-20	Booking Completes Successfully Without Entering ...	FLIGHT BOO...	CLOSED	Jul 2	HS
BFAT-21	Credit Card Month Field Accepts Invalid Values Bey...	FLIGHT BOO...	CLOSED	Jul 6	HS
BFAT-22	Passenger Information Displayed on Confirmation P...	FLIGHT BOO...	CLOSED	Jul 9	HS
BFAT-23	Booking Completes Successfully Without Entering ...	REPORTING ...	CLOSED	Jul 21	HS

Defect 1:

Name Field Accepts Special Characters Without Validation.

    Search      

Spaces /  BlazeDemo Flight Auto... /  BFAT-6 /  BFAT-19

Name Field Accepts Special Characters Without Validation

Description

Steps to Reproduce:

1. Open BlazeDemo application.
2. Search and select any flight.
3. Navigate to Purchase Flight page.
4. Enter only special characters in Name field (@#\$\$%^&+).
5. Fill all other mandatory fields.
6. Click Purchase Flight.

Expected Result:
Application should display a validation message and should not allow booking with special characters in the Name field.

Actual Result:
Application accepts special characters in the Name field and completes the booking successfully.

Details

Assignee
 Himanshu Sharma

Parent
 BFAT-6 Flight Search Automation

Due date
 Jun 24, 2026

Labels
None

Team
None

Start date
Jun 22, 2026

Defect 2

Booking Completes Successfully Without Entering City and State Details

    Search      

Spaces /  BlazeDemo Flight Auto... /  BFAT-9 /  BFAT-20

Booking Completes Successfully Without Entering City and State Details

Description

Environment:

- Operating System: Windows 11
- Browser: Google Chrome (Latest Version)
- Application: BlazeDemo Flight Booking Application

Precondition: User is on the Purchase Flight page after selecting a flight.

Steps to Reproduce:

1. Launch the BlazeDemo application.
2. Search and select any available flight.
3. Navigate to the Purchase Flight page.
4. Enter valid data in all mandatory fields except City and State.
5. Click on the Purchase Flight button.

Expected Result:
The application should validate the form and display an appropriate error message indicating that City field cannot be blank.

Details

Assignee
 Himanshu Sharma

Parent
 BFAT-9 Flight Booking Automation : Au

Due date
 Jul 2, 2026

Labels
None

Team
None

Start date
Jun 30, 2026

Defect 3

Credit Card Month Field Accepts Invalid Values Beyond valid Month Range

Jira

Search + Create Upgrade HS

Spaces / BlazeDemo Flight Aut... / BFAT-9 / BFAT-21

Credit Card Month Field Accepts Invalid Values Beyond Valid Month Range

+ ...

Description

Environment:

- Operating System: Windows 11
- Browser: Google Chrome (Latest Version)
- Application: BlazeDemo Flight Booking Application

Precondition: User is on the Purchase Flight page and ready to enter payment details.

Steps to Reproduce:

1. Launch the BlazeDemo application.
2. Search and select any available flight.
3. Navigate to the Purchase Flight page.
4. Enter valid passenger information.
5. In the Expiration Month field, enter an invalid value such as 13.
6. Fill in the remaining payment details with valid data.
7. Click on the Purchase Flight button.

Details

Assignee: HS Himanshu Sharma

Parent: BFAT-9 Flight Booking Automation : Au

Due date: Jul 6, 2026

Labels: None

Team: None

Start date: Jul 04, 2026

Defect 4

Passenger Information Displayed on Confirmation Page Does Not Match Entered Data.

Jira

Search + Create Upgrade HS

Spaces / BlazeDemo Flight Aut... / BFAT-9 / BFAT-22

Passenger Information Displayed on Confirmation Page Does Not Match Entered Data

+ ...

Description

Environment:

- Operating System: Windows 11
- Browser: Google Chrome (Latest Version)
- Application: BlazeDemo Flight Booking Application

Pre Condition: User has selected a flight and is on the Purchase Flight Page.

Steps to Reproduce:

1. Launch the BlazeDemo application.
2. Search and select any available flight.
3. Navigate to the Purchase Flight page. 4. Enter valid passenger details such as Name, Address, City, State, Zip Code, and Payment
4. Enter valid passenger details such as Name, Address, State, Zip Code, and Payment Information.
5. Click on the Purchase Flight button.
6. After booking is completed, navigate to the Confirmation Page.
7. Compare the passenger information displayed on the Confirmation Page with the data entered on

Details

Assignee: HS Himanshu Sharma

Parent: BFAT-9 Flight Booking Automation : Au

Due date: Jul 9, 2026

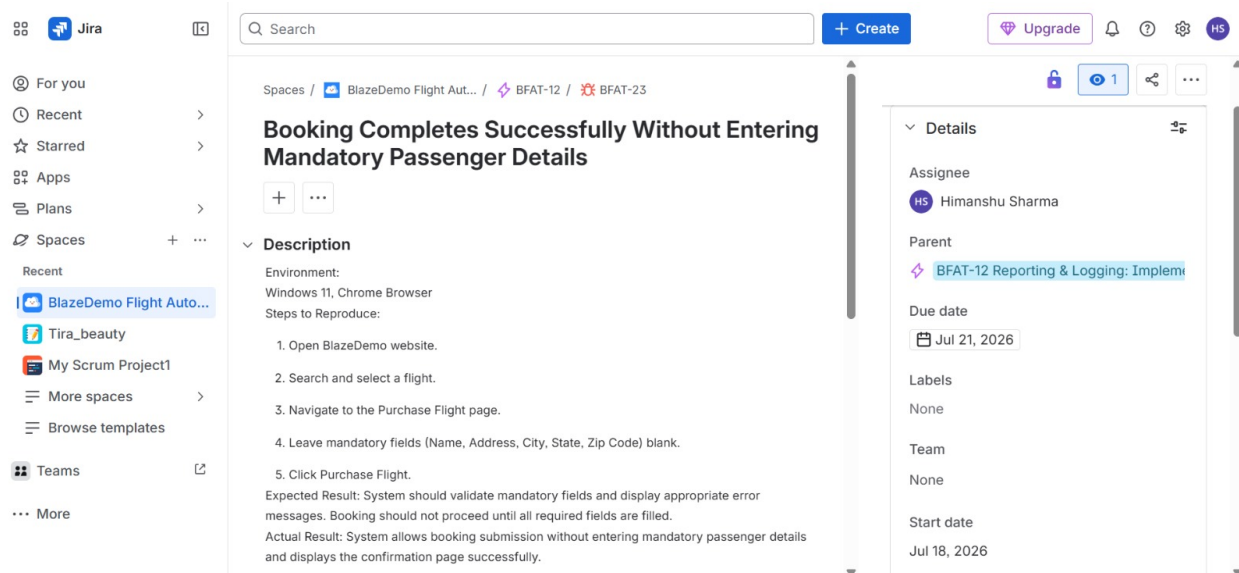
Labels: None

Team: None

Start date: Jul 07, 2026

Defect 5

Booking Completes Successfully Without Entering Mandatory Passenger Details.



All defects were documented and tracked through Jira.

Challenges Faced and Solutions

Challenge 1

Cross-browser execution and browser driver compatibility issues.

Solution

Implemented WebDriverManager and browser-specific configuration using TestNG parameters for Chrome and Firefox execution.

Challenge 2

Reading multiple test data records from Excel.

Solution

Implemented Apache POI and TestNG Data Providers to support Data-Driven Testing with multiple datasets.

Challenge 3

Detailed reporting and execution evidence requirement.

Solution

Integrated Extent Reports with TestNG Listeners and automatic screenshot capture for test execution.

Project Achievements

- Developed Selenium WebDriver Automation Framework
- Implemented Page Object Model (POM)
- Implemented Data-Driven Testing using Excel
- Integrated TestNG Framework
- Implemented Cross-Browser Testing (Chrome & Firefox)
- Generated Extent Reports for test execution
- Implemented Automatic Screenshot Capture
- Externalized configuration using config.properties
- Integrated Jenkins for Continuous Integration (CI)
- Managed source code using Git and GitHub

Conclusion

The BlazeDemo Flight Booking Automation Framework successfully automates the complete flight booking process using Selenium WebDriver, Java, and TestNG. The framework follows the Page Object Model (POM) architecture to ensure code reusability and maintainability. It supports Data-Driven Testing using Excel and provides detailed execution reports through Extent Reports and TestNG. Cross-browser testing, screenshot capture, and Jenkins integration enhance the reliability and efficiency of the framework. Overall, the project demonstrates industry-standard automation testing practices and provides a scalable and maintainable solution for web application testing.